

GovMatch — Project Documentation

Version: 1.0 (MVP) **Last updated:** July 2026 **Status:** Deployed, functional MVP — pre-launch

1. Executive Summary

GovMatch is a personalized government job discovery platform built specifically to address a gap in the Indian government job market: existing platforms (Sarkari Result, FreeJobAlert, SarkariExam) are generic listing aggregators with no profile-based matching, no eligibility filtering, and no application tracking. GovMatch's core differentiator is a **matching engine** that scores every open job posting against a user's actual profile (education, age, category), rather than showing every listing to every visitor.

The MVP is now feature-complete for a single-user validation phase: full backend (database, API, admin CMS), real authentication, a working matching algorithm, and a public-facing site with search, job listings, and personal dashboards — deployed and live.

2. Original Requirement

2.1 Problem Statement

As a BE CSE student, government job opportunities relevant to a specific technical background are:

- Infrequent and scattered across many official sources (SSC, UPSC, RRB, PSU career pages, state PSCs)
- Difficult to discover because existing "Sarkari" job sites list everything for everyone, with no filtering by degree/branch relevance
- Hard to track once discovered (no centralized way to see deadlines, application status, or exam prep in one place)

2.2 Initial Feature Requirements (as scoped)

1. Showcase recent/upcoming government jobs
2. User profile creation (personal + educational information)
3. Personalized job recommendations based on profile/interest
4. Education-to-job match percentage
5. A paid mock-test section (deferred — not built in this MVP phase)

2.3 Competitive Positioning

Research confirmed no existing Indian platform combines **LinkedIn-style personalization**

with **government job aggregation**. The closest official equivalent (National Career Service / NCS) is a basic government-run job board with no matching intelligence. This validated the product's core differentiation: *eligibility-aware, personalized matching*, not just another listing site.

3. What We Built — Full Inventory

3.1 Database Layer (PostgreSQL via Supabase, Prisma ORM)

11 core tables, modeled to reflect a real structural insight: a single recruitment **Notification** (e.g. "RRB Technician 2026") can contain multiple **Posts** (e.g. "Technician Grade III", "Junior Engineer"), each with independently different eligibility criteria (age, qualification, vacancies, pay scale). Flattening this into one row per notification — which is how most competitor sites work — would have made accurate matching impossible.

Table	Purpose
<code>Organization</code>	Recruiting bodies (SSC, RRB, IBPS, UPSC, IAF, Navy, DSSSB, ISRO, IOCL, etc.)
<code>State</code>	Indian states, for state-govt job filtering
<code>Category</code>	Job category taxonomy (Central Govt, Banking, Railway, Teaching, Defence, etc.)
<code>Qualification</code>	Education levels (10th, 12th, ITI, Diploma, Graduate, B.Tech, PG) with a sortable <code>level</code> for eligibility comparison
<code>Notification</code>	The recruitment drive — title, org, dates, status, official link
<code>Post</code>	Individual role within a notification — vacancies, age range, qualification requirement, pay scale
<code>AdmitCard</code> / <code>Result</code> / <code>AnswerKey</code> / <code>Document</code>	Lifecycle content tied back to a <code>Notification</code>
<code>User</code>	Account record — email, name, auth provider, password hash
<code>UserProfile</code>	DOB, category, qualification, degree, percentage — the data the matching engine reads
<code>SavedJob</code> / <code>Application</code>	User-tracked jobs and application status (Kanban: Applied → Exam Scheduled → Result Awaited → Selected)
<code>Account</code> / <code>Session</code> / <code>VerificationToken</code>	NextAuth.js-required tables for OAuth and session management

Why Prisma + PostgreSQL (via Supabase):

- Prisma gives type-safe database access directly from TypeScript, so the same types flow from the database through the API to the frontend with no manual re-typing
- Supabase provides a managed Postgres instance with zero local setup, a generous free tier, and built-in connection pooling (needed for serverless deployment on Vercel, where each request can spin up a new connection)
- Postgres (relational) was the right fit over a NoSQL option because the domain is inherently relational — a Post belongs to a Notification which belongs to an Organization, and a User's Applications reference Notifications. Enforcing these relationships at the database level (foreign keys, cascading deletes) prevents an entire class of data-integrity bugs that a document database would leave to application code to manage.

3.2 Backend API (Next.js API Routes)

Full REST CRUD for every table:

```
GET/POST      /api/{entity}
GET/PATCH/DELETE /api/{entity}/{id}
```

Covering: `organizations`, `states`, `categories`, `qualifications`, `notifications`, `posts`, `admit-cards`, `results`, `answer-keys`, `documents`.

Plus user-scoped routes:

```
GET/PATCH      /api/users/{userId}/profile
GET/POST       /api/users/{userId}/saved-jobs
DELETE         /api/users/{userId}/saved-jobs/{notificationId}
GET/POST       /api/users/{userId}/applications
PATCH/DELETE  /api/users/{userId}/applications/{id}
GET            /api/users/{userId}/matches
```

Why Next.js API routes instead of a separate backend (e.g. NestJS): A separate backend was considered early on, but for a solo-built MVP, colocating the API inside the Next.js app eliminates an entire category of overhead: no CORS configuration, no second deployment target, no duplicated environment/auth setup, and one Vercel deploy ships both frontend and backend together. This trades away some separation-of-concerns purity for significantly faster iteration speed — the right tradeoff at this stage. It can be split into a dedicated service later if/when the API needs to scale independently or serve additional clients (e.g. a mobile app).

3.3 Matching Engine

A dedicated scoring function (`features/matching/engine.ts`) computes a 0-100 match score for a user profile against a specific `Post`:

- **Qualification match (60% weight):** user's qualification level must meet or exceed the post's required level
- **Age match (40% weight):** user's computed age (from DOB) must fall within the post's min/max age range
- Returns both a numeric score and a `breakdown` object (which factors matched/didn't), plus a boolean `eligible` flag used to visually flag "eligible" vs "close match" listings

Why a weighted rules engine instead of ML from day one: With limited seed data, a machine-learning matching model would have nothing meaningful to train on and would add complexity without adding accuracy. A transparent, deterministic rules engine is also more trustworthy to the end user — "why did I get 85%?" has a clear, explainable answer, which matters for a platform dealing with people's career decisions. This can evolve into a

more sophisticated scoring model (weighted by more factors: state, category reservation, experience) once there's real usage data to justify it.

3.4 Admin Panel (CMS)

Password-gated (`/admin`), protected by Next.js proxy/middleware checking a session cookie) full CRUD interface for every table — list views, create forms, edit forms, delete with confirmation. Built on three reusable generic components (`DataTable`), (`FormField`), (`RecordForm`) driven by per-entity config objects, rather than hand-building a bespoke UI for each of the 11 tables.

Why a generic component pattern instead of one-off admin pages: Given the number of tables, building each admin page from scratch would have meant repeating the same list/form/validation logic eleven times. The config-driven pattern means adding a twelfth table in the future is a ~20-line config file, not a new set of components.

3.5 Authentication

NextAuth.js (Auth.js v5) with:

- **Credentials provider** (email + bcrypt-hashed password)
- **Google OAuth provider** (configured, pending real production credentials)
- **Prisma adapter** for session/account persistence
- Signup captures full profile data (DOB, category, qualification, degree) in a single form, so a new user's matching engine has data to work with immediately rather than starting empty

Why NextAuth instead of building auth from scratch: Authentication is security-sensitive, solved territory. NextAuth handles password hashing conventions, session/cookie security, CSRF protection, and OAuth flow correctly out of the box — reimplementing this is pure risk with no product upside.

3.6 Public-Facing Frontend

- **Homepage** — two full variants: logged-out (marketing hero, category browsing, latest jobs) and logged-in (personalized search hero with live match score, quick stats, recommended jobs sidebar, profile strength indicator)
- **Search & job listing** (`/jobs`) — real-time search/filter against live data, with match-percentage badges for logged-in users
- **Job detail pages** — full posting info, per-post breakdown, Save/Mark-as-Applied actions
- **Admit Cards / Results / Exams** pages — pulling live data from their respective tables
- **Profile page** — user-editable education/eligibility data
- **Saved Jobs and Applications Tracker** (Kanban-style: Applied / Exam Scheduled / Result Awaited / Selected)

Why Next.js (App Router) + Tailwind CSS:

- Next.js's App Router gives server-side rendering for SEO-critical pages (job listings need to rank on Google, same as competitor sites do) while still supporting client-side interactivity where needed (search, forms, dashboards)
- Route groups (`((marketing))`), (`(auth)`), (`((dashboard))`), (`((public-data))`) let different sections of the site have different layouts (navbar+footer vs. sidebar vs. no-chrome auth pages) without duplicating routing logic
- Tailwind CSS enables rapid, consistent UI development directly from design tokens (colors, spacing, radius) defined once and reused everywhere, avoiding CSS drift across dozens of components

3.7 Component Architecture

A deliberate separation was enforced partway through the build after recognizing an early structural mistake:

- `components/ui/` — pure, reusable primitives (Button, Card, Badge) with zero business logic
- `components/jobs/`, `components/exams/`, `components/shared/` — reusable domain components (e.g. `JobListItem`) shared across the homepage, `/jobs` listing, and dashboard — not duplicated per page
- `components/sections/` — page-specific composition/orchestration (Hero, AuthCard) that assembles the above into a specific page layout
- `features/{domain}/` — all data-fetching, hooks, and business logic, separated from UI components entirely

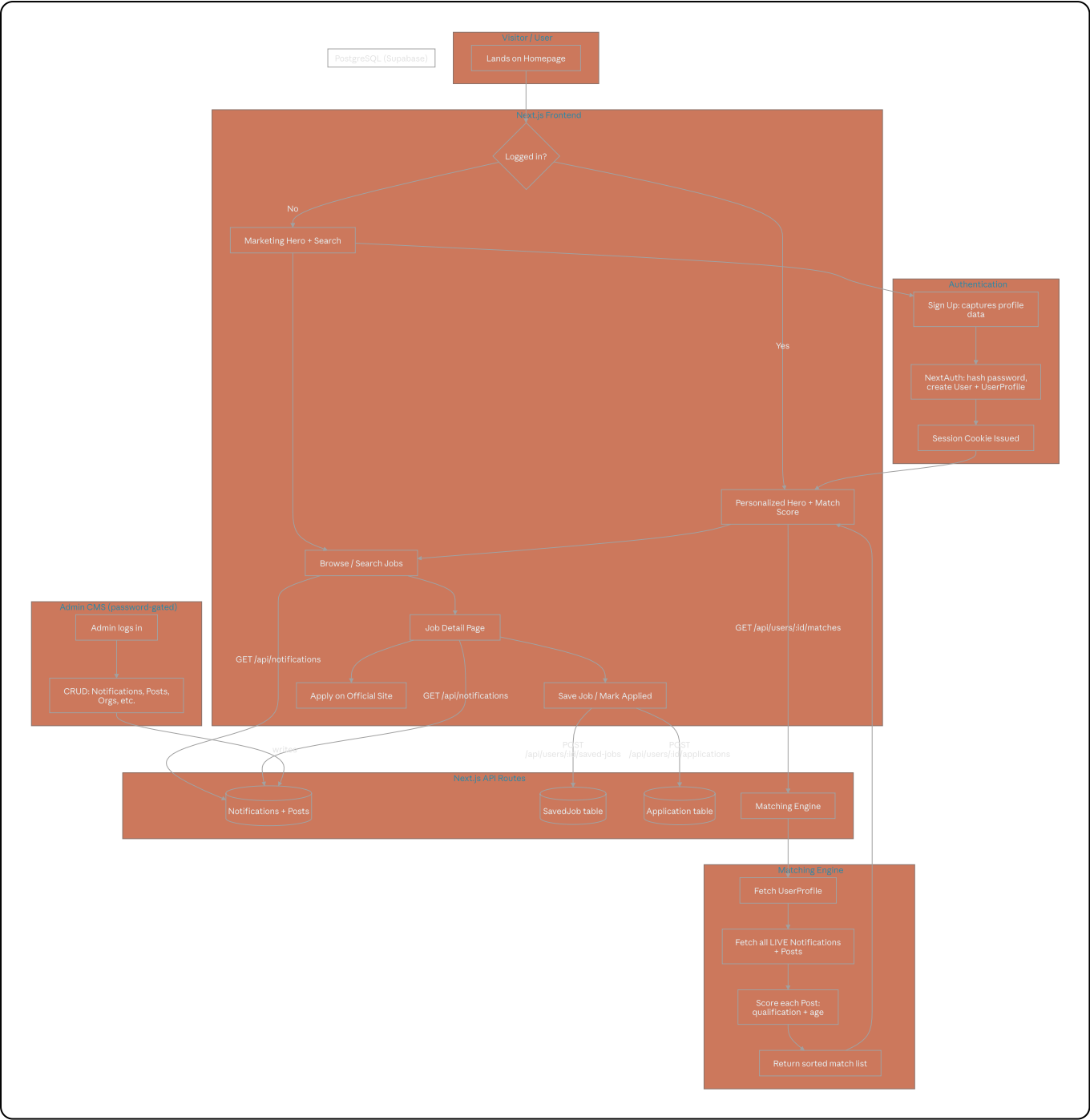
Why this structure: the initial instinct was to nest everything under a `components/home/` folder, but this was corrected once it became clear that components like job-list rows and category cards needed to be reused on multiple pages (home, `/jobs`, dashboard), not just the homepage. Separating "reusable domain UI" from "page-specific layout" avoided duplicating the same component in three places as the site grew.

4. Technology Stack Summary

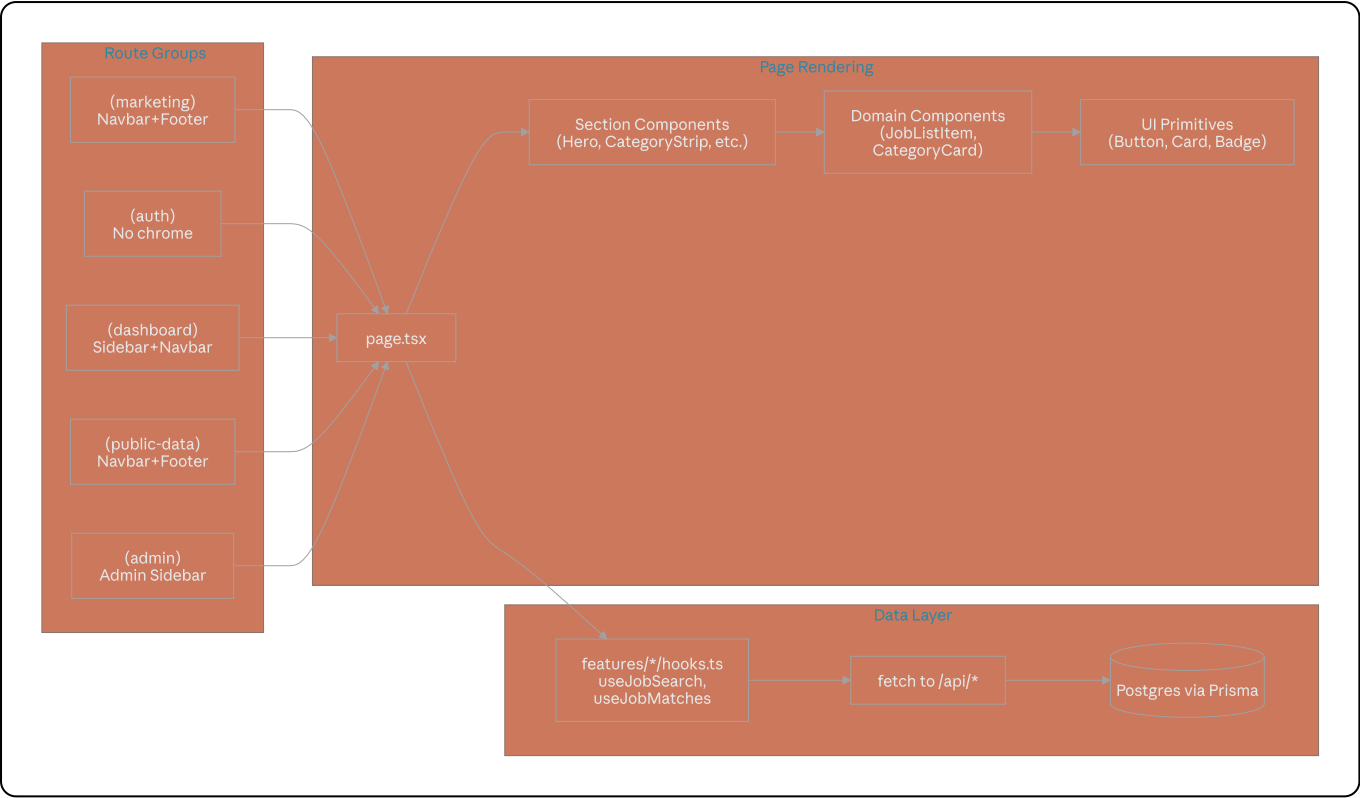
Layer	Choice	Why
Frontend framework	Next.js 16 (App Router)	SSR for SEO, unified frontend+API, React Server Components
Styling	Tailwind CSS v4	Design-token-driven, fast iteration, no CSS drift
Language	TypeScript	Type safety end-to-end, from DB schema to UI props

Layer	Choice	Why
Database	PostgreSQL (Supabase)	Relational integrity for inherently relational domain data
ORM	Prisma 7	Type-safe queries, migrations, schema-as-source-of-truth
Auth	NextAuth.js (Auth.js v5)	Solved, secure, supports both credentials and OAuth
Hosting	Vercel	Zero-config Next.js deploys, integrates with GitHub
Icons	Lucide React	Consistent, tree-shakeable icon set

5. End-to-End System Flow

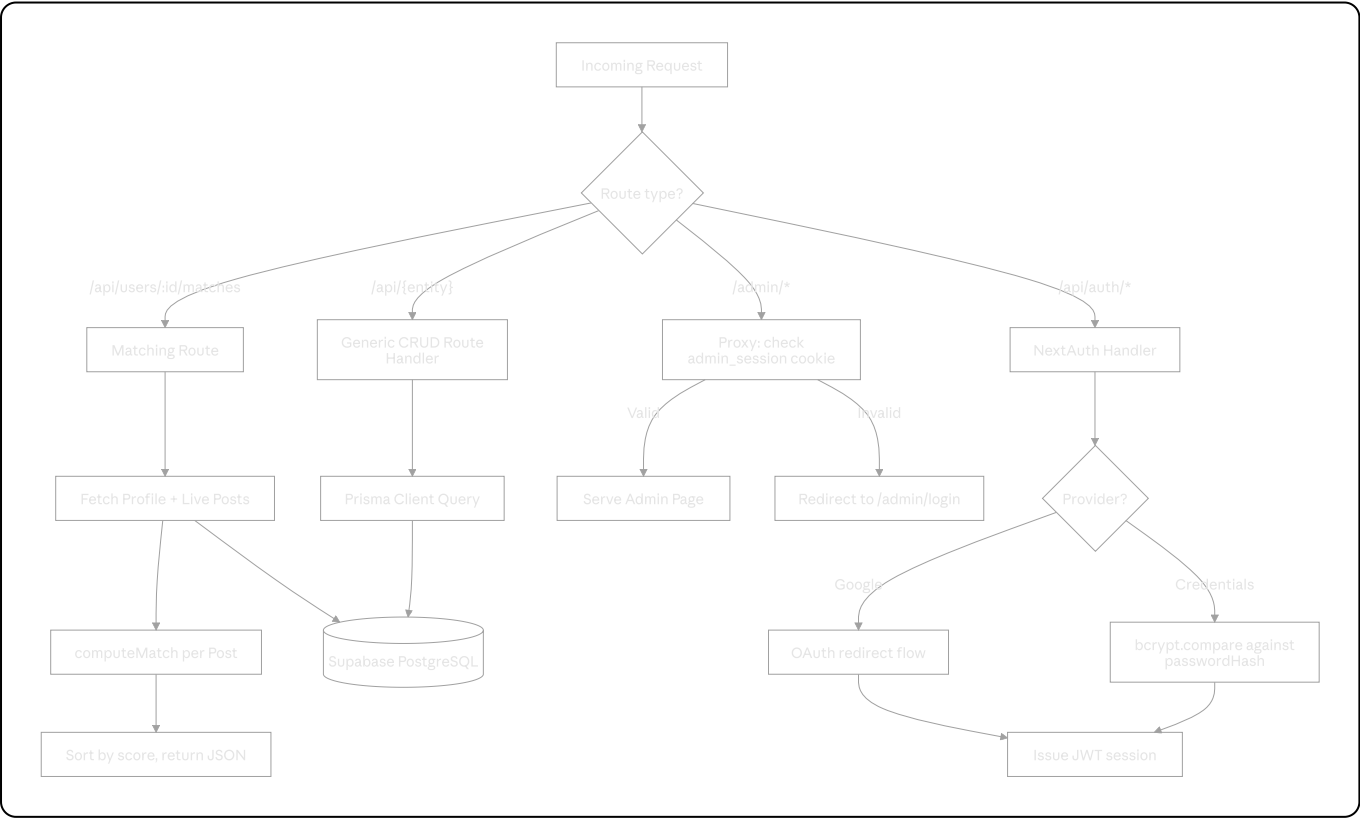


6. Frontend Process Flow (Detailed)

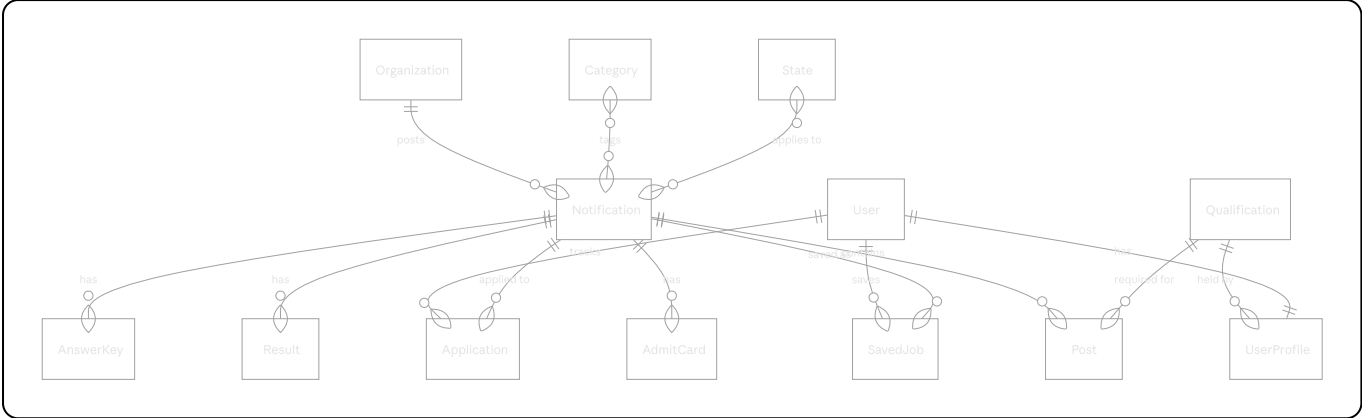


Why this flow: pages never call `fetch()` directly — all data access goes through a `features/{domain}/hooks.ts` hook. This means swapping mock data for real API calls (which happened multiple times during this build) only required changing one file per domain, not every component that displayed that data.

7. Backend Process Flow (Detailed)



8. Database Entity Relationship (Simplified)



9. What's Left / Not Yet Built

Area	Status	Notes
Google OAuth	Configured, placeholder credentials	Needs real Google Cloud Console credentials + production redirect URI
Content pipeline	Manual seed data only (9 notifications)	No automated scraping yet — was flagged early as the hardest unsolved problem in this space; needs RSS/scrapper build per official source
Mock test module	Not built	Originally scoped as a paid feature; deferred entirely in this MVP
Payments	Not built	Needed once mock tests or any paid tier is introduced (Razorpay recommended for India)
Notifications (push/email/SMS)	Not built	"Smart Alerts" is currently just UI copy, no backend trigger system
GATE Calculator	Placeholder page only	Mentioned in nav, not implemented
Study Zone / Resources	Placeholder pages only	"Coming Soon" stubs
Mobile responsiveness	Partially verified	Desktop-first build; needs a dedicated mobile QA pass
Admin role management	Single shared password	No per-admin accounts or granular permissions

Area	Status	Notes
Rate limiting / abuse protection	Not implemented	APIs are open to any authenticated (or in some cases unauthenticated) request volume
Automated testing	None	No unit/integration/e2e tests exist yet
Category/State-based matching	Not in scoring engine	Matching engine currently only weighs qualification + age, not category reservation or location

10. Recommended Next Steps (Priority Order)

1. **Content pipeline** — without a real, ongoing stream of job postings, nothing else matters; this was identified as the core hard problem from the start of the project
2. **Mobile QA pass** — majority of the target audience will access via phone
3. **Google OAuth real credentials** — low effort, meaningfully reduces signup friction
4. **Expand matching factors** — add category (reservation) and state to the scoring engine, since these are commonly used real eligibility filters
5. **Basic notification system** — even simple email alerts on new matching jobs would deliver on the "Smart Alerts" promise already shown in the UI
6. **Automated tests** — at minimum, cover the matching engine's scoring logic and the signup/auth flow, since these are the highest-consequence code paths
7. **Mock test module + payments** — once there's a validated, retained user base, not before

11. Future Improvement Ideas (Beyond MVP)

- **ML-enhanced matching** once sufficient usage/outcome data exists (did users who got X% match actually apply/get selected?)
- **Peer discussion per exam listing** (cutoff trends, expected vacancy discussion) — flagged early as a community/trust-building feature competitors don't offer
- **Document vault** — store verified certificates once, reuse across applications
- **B2B lead feed** — surfacing filtered, consenting candidate pools to coaching institutes/PSUs as an additional revenue stream beyond consumer subscriptions
- **GATE score predictor** — ties into the large volume of GATE-gated PSU recruitment specifically relevant to engineering graduates
- **Native mobile app** — push notifications are meaningfully more reliable via a real app than web push, and this audience is mobile-first

12. Key Architectural Decisions — Quick Reference

Decision	Alternative considered	Why this choice won
Notification → Post (one-to-many)	Flat job listing per row	Real eligibility (age/qualification/vacancies) varies per post within one drive; flattening would break matching accuracy
Next.js API routes	Separate NestJS backend	Faster solo iteration, one deploy target, no CORS/auth duplication
Rules-based matching engine	ML model from day one	No training data yet; transparency/explainability matters for career decisions
NextAuth.js	Custom auth	Security-sensitive, solved problem — not worth reinventing
Prisma + PostgreSQL	Firebase/NoSQL	Domain is inherently relational; foreign-key integrity prevents data bugs
Config-driven generic admin components	Hand-built page per table	11 tables would mean 11x duplicated CRUD logic otherwise
Domain components separated from page sections	Everything nested under <code>components/home/</code>	Components like job rows needed reuse across home, /jobs, and dashboard